φ-Calculus: Object-Oriented Formalism

Ver: 0.0.0

YEGOR BUGAYENKO, Huawei, Russia
MAXIM TRUNNIKOV, Huawei, Russia

Object-oriented programming (OOP) is one of the most popular paradigms used for building software systems[1]. However, despite its industrial and academic popularity, OOP is still missing a formal apparatus similar to λ-calculus, which functional programming is based on. A number of attempts were made to formalize OOP, but none of them managed to cover all the features available in modern OO programming languages, such as C++ or Java. We have made yet another attempt and created φ-calculus. This paper does not demonstrate the practical use or effect of φ-calculus but merely explains it.

Additional Key Words and Phrases: Object-Oriented Programming, Object Calculus

1 Introduction

Object-oriented programming has become the dominant paradigm for software development, with languages such as Java, C++, Python, and C# being among the most widely used in industry [**? ? ? ? ? ?**]. Despite this prevalence, the theoretical foundations of OOP remain less developed than those of functional programming [**? ?**], which has long benefited from the λ-calculus [**? ?**] as a formal basis. The absence of a comparable formalism for OOP hinders rigorous reasoning about object-oriented programs, complicates the development of verified compilers and static analyzers, and limits the potential for principled language design [**?**].

Several attempts have been made to formalize object-oriented concepts. **?**] proposed an imperative object calculus, while **?**] introduced Featherweight Java as a minimal core calculus. However, these formalisms either focus on specific language features or omit constructs commonly found in modern OO languages. As **?**] observed, the development of object-based programming languages has suffered from the lack of any generally accepted formal foundations.

This paper presents φ-calculus, a calculus designed to serve as a formal foundation for object-oriented programming. The calculus is built around a single primitive—the object formation—which encapsulates attributes, data, and functions. Objects interact through application, which attaches expressions to attributes, and dispatch, which retrieves attached expressions.

The contributions of this paper are as follows:

- A formal syntax for φ-calculus defined by a context-free grammar (Section 3).
- Rigorous definitions of the core constructs (Section 4).
- A family of semantic operators (Section 5).
- A reference interpreter that runs φ-expressions[2].

[1] LaTeX sources of this paper are maintained in the REPOSITORY public GitHub repository, the rendered version is 0.0.0.
[2] https://github.com/objectionary/phino

The remainder of this paper is organized as follows. Section 2 provides an informal overview of the calculus. Section 3 presents the syntax as a context-free grammar. Section 4 defines the foundational concepts. Section 5 introduces the semantic operators. Section 7 suggests directions of future studies. Finally, Section 6 discusses related work.

2 Informal Overview

An object *formation* is a collection of *attributes*, which are uniquely named pairs, for example:

$$[\![\texttt{price} \mapsto \varnothing, \texttt{color} \mapsto [\![\Delta \mapsto \texttt{FF-C0-CB}]\!]]\!]. \tag{1}$$

This formation has two attributes `price` and `color`. The formation is an *abstract* because the `price` attribute is *void*, i.e. there is nothing *attached* to it. The `color` attribute is attached to another formation with one Δ-*asset*, which is attached to *data* (three bytes).

A *full application* of a pair—an attribute and an *expression*—to an abstract *results* in a new *closed* formation, for example:

$$[\![\texttt{x} \mapsto \varnothing]\!](\texttt{y} \mapsto b) \rightsquigarrow [\![\texttt{x} \mapsto b]\!]. \tag{2}$$

A *partial* application results in a new abstract, for example:

$$[\![\texttt{x} \mapsto \varnothing, \texttt{y} \mapsto \varnothing]\!](\texttt{x} \mapsto b) \rightsquigarrow [\![\texttt{x} \mapsto b, \texttt{y} \mapsto \varnothing]\!].$$

A formation may be *dispatched* from another formation with the help of *dot notation* where the right side *accesses* the left side, for example:

$$[\![\texttt{x} \mapsto \xi.\texttt{p}(\texttt{t} \mapsto b).\texttt{y}, \texttt{p} \mapsto [\![\texttt{t} \mapsto \varnothing, \texttt{y} \mapsto \xi.\texttt{t}]\!]]\!].\texttt{x} \rightsquigarrow b. \tag{3}$$

The leftmost symbol ξ denotes the formation itself; the following `.p` part retrieves the formation attached to the attribute `p`; then, the application $(\texttt{t} \mapsto b)$ makes a copy of the formation; finally, the `.y` part retrieves formation b.

Attributes are *immutable*, i.e. an application to a formation, where the attribute is already attached, results in a *terminator* denoted as $\bot$ (runtime error), for example:

$$[\![\texttt{x} \mapsto b_1]\!](\texttt{x} \mapsto b_2) \rightsquigarrow \bot.$$

An expression may be textually reduced, for example:

$$\begin{aligned}[\![\texttt{x} \mapsto \varnothing, \texttt{y} \mapsto \varnothing]\!](\texttt{x} \mapsto b_1)(\texttt{y} \mapsto b_2) &\rightsquigarrow \\ \rightsquigarrow [\![\texttt{x} \mapsto b_1, \texttt{y} \mapsto \varnothing]\!](\texttt{y} \mapsto b_2) &\rightsquigarrow \\ \rightsquigarrow [\![\texttt{x} \mapsto b_1, \texttt{y} \mapsto b_2]\!].\end{aligned}$$

The formation on the left is reduced to the formation on the right in two reduction steps. Some expressions may be in *normal form*, which means no further applicable reductions.

A formation is called a *decorator* if it has the φ-attribute with an expression attached to it, known as a *decoratee*. An attribute dispatched from a decorator reduces to the same attribute dispatched from the decoratee, if the attribute is not present in the decorator, for example:

$$[\![\varphi \mapsto [\![\texttt{x} \mapsto b]\!]]\!].\texttt{x} \rightsquigarrow b.$$

$$
\begin{aligned}
\langle \text{Program} \rangle &\to \Phi \mapsto \langle \text{Formation} \rangle \\
\langle \text{Expression} \rangle &\to \langle \text{Formation} \rangle \mid \langle \text{Application} \rangle \mid \langle \text{Dispatch} \rangle \mid \langle \text{Locator} \rangle \mid \bot \\
\langle \text{Formation} \rangle &\to [\![\langle \text{Binding} \rangle]\!] \\
\langle \text{Application} \rangle &\to \langle \text{Expression} \rangle \ (\ \langle \text{A-Pair} \rangle \ ) \\
\langle \text{A-Pair} \rangle &\to \langle \tau\text{-Pair} \rangle \mid \langle \alpha\text{-Pair} \rangle \\
\langle \text{Dispatch} \rangle &\to \langle \text{Expression} \rangle \ . \ \langle \text{Attribute} \rangle \\
\langle \text{Locator} \rangle &\to \Phi \mid \xi \\
\langle \text{Binding} \rangle &\to \langle \text{Pair} \rangle \ \langle \text{Bindings} \rangle \mid \epsilon \\
\langle \text{Bindings} \rangle &\to , \ \langle \text{Pair} \rangle \ \langle \text{Bindings} \rangle \mid \epsilon \\
\langle \text{Pair} \rangle &\to \langle \varnothing\text{-Pair} \rangle \mid \langle \tau\text{-Pair} \rangle \mid \langle \Delta\text{-Pair} \rangle \mid \langle \lambda\text{-Pair} \rangle \\
\langle \varnothing\text{-Pair} \rangle &\to \langle \text{Attribute} \rangle \mapsto \varnothing \\
\langle \tau\text{-Pair} \rangle &\to \langle \text{Attribute} \rangle \mapsto \langle \text{Expression} \rangle \\
\langle \alpha\text{-Pair} \rangle &\to \langle \text{Alpha} \rangle \mapsto \langle \text{Expression} \rangle \\
\langle \Delta\text{-Pair} \rangle &\to \Delta \ \text{-}\!\!\rightarrow \langle \text{Data} \rangle \\
\langle \lambda\text{-Pair} \rangle &\to \lambda \ \text{-}\!\!\rightarrow \langle \text{Function} \rangle
\end{aligned}
$$

Fig. 1. Syntax as a context-free grammar, in BNF.

A formation may have data attached only to its Δ-asset, for example:

$$[\![\mathtt{x} \mapsto [\![\Delta \ \text{-}\!\!\rightarrow \mathtt{00\text{-}2A}]\!]]\!].$$

A formation may have a *function* attached to its λ-asset. Such a formation is referred to as an *atom*. An atom may be *morphed* to another formation by evaluating its function, for example:

$$\frac{\vdash \mathtt{Sqrt}(b,\ u,\ s) \to \langle [\![\Delta \ \text{-}\!\!\rightarrow \sqrt{\mathbb{D}(b.\mathtt{x},\ u,\ s)}]\!],\ s \rangle}{[\![\lambda \ \text{-}\!\!\rightarrow \mathtt{Sqrt},\ \mathtt{x} \mapsto 256]\!] \to 16}. \tag{4}$$

There is also a *dataization* function $\mathbb{D}(b, u, s)$ that normalizes its first argument and then either returns the data attached to the Δ-asset of the normal form or morphs the atom into a formation and dataizes it (recursively), for example:

$$\mathbb{D}([\![\mathtt{x} \mapsto 42]\!].\mathtt{x} \rightsquigarrow 42,\ u,\ s) \to 42.$$

A program is a formation, called *Universe*, that is attached to the Φ attribute of nowhere:

$$\Phi \mapsto [\![\Delta \ \text{-}\!\!\rightarrow \mathtt{CA\text{-}FE}]\!].$$

The dataization of universe is the evaluation of the program.

3 Syntax

The syntax of a program is defined by BNF in Fig. 1, where the starting symbol is $\langle \text{Program} \rangle$.

Besides the literals mentioned in the grammar in orange, the alphabet includes three non-terminals that rewrite to terminals as follows:

Table 1. Syntax sugar.

Syntax sugar	Its more verbose equivalent
$e(\tau_1 \mapsto e_1, \tau_2 \mapsto e_2, \ldots)$	$e(\tau_1 \mapsto e_1)(\tau_2 \mapsto e_2)\ldots$
$e(e_0, e_1, \ldots)$	$e(\alpha_0 \mapsto e_0, \alpha_1 \mapsto e_1, \ldots)$
$\tau_1(\tau_2, \tau_3, \ldots) \mapsto [\![B]\!]$	$\tau_1 \mapsto [\![\tau_2 \mapsto \varnothing, \tau_3 \mapsto \varnothing, \ldots, B]\!]$
$\tau_1 \mapsto \tau_2$	$\tau_1 \mapsto \xi.\tau_2$
$[\![B]\!]$	$[\![B, \rho \mapsto \varnothing]\!]$ if $\rho \notin B$
"你好"	Φ.`string`(Φ.`bytes`($[\![\Delta \mapsto$ `E4-BD-A0-E5-A5-BD`$]\!]$)) (UTF-8 string)
42	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `40-45-00-00-00-00-00-00`$]\!]$)) (eight bytes per integer)
3.14	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `40-09-1E-B8-51-EB-85-1F`$]\!]$)) (eight bytes per number with a floating point)
$\{e\}$	$\Phi \mapsto e$

- ⟨Attribute⟩: either 1) Greek letter φ, 2) Greek letter ρ, or 3) a string of lowercase English letters possibly with dashes inside, e.g. "`price`" or "`a-car`";
- ⟨Data⟩: a sequence of bytes in hexadecimal format, e.g. "`EF-41-5C`" is a sequence of three bytes, "`42-`" is a one-byte sequence (with a trailing dash to avoid confusion with integers), and "`--`" (double dash) is an empty sequence of bytes;
- ⟨Function⟩: a string of English letters and numbers where the first symbol is an uppercase letter, e.g. "`Sqrt`" or "`F1`";
- ⟨Alpha⟩: a Greek letter α with a non-negative whole-number index, e.g. α_2.

Table 1 shows all possible syntax sugar.

4 Foundations

In this section, we introduce the core concepts of the calculus. The purpose of this section is to define the entities that constitute the calculus—such as expressions, formations, and attributes—together with their structural relationships. The definitions given here are descriptive rather than operational. They explain what expressions are, not how they are evaluated or transformed.

Definition 4.1 (Expression). An **expression**, ranged over $\mathcal{E}$ by e_i, is a grammatical construct obeying the syntax of Fig. 1.

Definition 4.2 (Attribute). An **attribute**, ranged over $\mathcal{T}$ by τ_i, is an identifier to which either $\varnothing$ or an expression may be attached.

Definition 4.3 (Alpha). An **alpha**, ranged over $\mathcal{H}$ by η_i, is a positional identifier.

Definition 4.4 (Void vs. Attached). An attribute is **void** if $\varnothing$ is attached to it, otherwise it is an **attached** attribute.

Definition 4.5 (Data). **Data**, ranged over $\mathcal{D}$ by δ_i, is a possibly empty sequence of 8-bit bytes.

Definition 4.6 (State). A **state** of evaluation s_i ranges over $\mathcal{S}$, where s_0 is an empty state.

Definition 4.7 (Function). A **function** is a total mapping $\mathcal{B} \times \mathcal{B} \times \mathcal{S} \to \mathcal{E} \times \mathcal{S}$ that deterministically maps formation, universe, and state to an expression and a new state.

Definition 4.8 (Asset). An **asset** is an identifier to which either data (denoted as Δ-asset) or function (denoted as λ-asset) is attached.

Definition 4.9 (Binding). A **binding**, ranged over $\mathcal{G}$ by B_i, is a possibly empty sequence of key-value pairs, where all keys are unique.

Definition 4.10 (Formation). An object **formation**, denoted as $[\![B]\!]$, ranged over $\mathcal{B}$ by b_i, is a binding.

If $b = [\![B]\!]$, then the predicate $k \in b$ holds if the key k is present B.

We introduce the term "formation" rather than using the more traditional term "construction" because the latter generally implies the presence of a class from which an object is being constructed or instantiated. Instead, formation is closer to the creation of a prototype, which may either be used "as is" or copied.

The following is an example of a formation with four pairs, where the first one is an asset attached to data, while the other three are attributes attached to expressions:

$$[\![\Delta \mapsto \texttt{00-2A}, \texttt{b} \mapsto b_2(\alpha_0 \mapsto b_3).\texttt{bar}, \texttt{a} \mapsto [\![\lambda \mapsto \texttt{Sqrt}]\!], \texttt{foo} \mapsto \bot]\!]. \tag{5}$$

The arrow $\mapsto$ denotes an attachment of an expression (right-hand side) to an attribute (left-hand side). The arrow $\mapsto$ denotes an attachment of data or function to an asset.

Definition 4.11 (Domain). A **domain** of formation b, denoted as $\bar{b}$, is a sequence of all attributes of b, excluding assets.

The domain of the formation in Eq. (5) is $\langle \texttt{b}, \texttt{a}, \texttt{foo} \rangle$. Assets Δ and λ do not belong to the domain.

Definition 4.12 (Program). A **program**, also known as *Universe*, is a formation attached to Φ.

Definition 4.13 (Terminator). The **terminator**, denoted as $\bot$, is an expression that signals an error.

Definition 4.14 (Abstract). An **abstract** is a formation with at least one void attribute.

Equation (1) is an example of a formation that is an abstract, while the expression attached to its `color` attribute is not an abstract.

A formation that is not an abstract may be called a *closed* formation.

Definition 4.15 (Application). An **application**, denoted as $e_1(\tau \mapsto e_2)$ or $e_1(\eta_i \mapsto e_2)$, means an attempt to attach e_2 (the "argument") to either the τ attribute of e_1 (the "subject") or the i-th attribute of e_1.

Equation (2) demonstrates application, where $\texttt{a} \mapsto b_2$ is applied to an abstract $[\![\texttt{a} \mapsto \varnothing]\!]$. The application creates a new formation $[\![\texttt{a} \mapsto b_2]\!]$, while the existing formation remains intact.

The formation in Eq. (1) is an abstract, because its attribute `price` is void. The formation in Eq. (2) was an abstract before the application, but the formation created by the application is closed since its attribute `a` is not void (attached to b_2).

Even though $\varnothing$ being attached to an attribute of a formation resembles NULL references, there is a significant difference: in φ-calculus, void attributes may be attached to expressions only once, while any further reattachments are prohibited.

In $b(\alpha_i \mapsto e)$ application, e must be attached to the i-th attribute of b.

Definition 4.16 (Dispatch). A **dispatch**, denoted as $e.\tau$, where e is the "subject," means an attempt to retrieve what is attached to τ in the formation to which e may be transformed.

Definition 4.17 (Atom). An **atom** is a formation with a function attached to its λ-asset.

Equation (4) demonstrates an atom with a function that calculates the square root of a number, which it retrieves from the Δ-asset of $b.\alpha_0$ with the help of the morphing function (Section 5.5). The implementation of functions is outside the scope of φ-calculus: they may be implemented, for example, in λ-calculus or a programming language such as Java or C++.

Definition 4.18 (Decoration). **decoration** is a mechanism of extending a formation ("decoratee") by attaching it to the φ-attribute of a formation ("decorator"), which makes attributes of the decoratee retrievable from the decorator, unless the decorator has its own attributes with the same names.

Definition 4.19 (Parent). Attaching expression e to the ρ attribute of formation b means setting the **parent** of b to e.

The presence of "parent" in each formation is essential for the coordination of inner formations after dispatch. Consider the following abstract with an inner formation:

$$\Big\{\texttt{x} \mapsto [\![\texttt{a} \mapsto \varnothing, \texttt{next} \mapsto [\![\varphi \mapsto \xi.\rho.\texttt{a.plus}(\texttt{1}), \rho \mapsto \Phi.\texttt{x}]\!], \texttt{k} \mapsto \xi.\texttt{x}(\texttt{42})\Big\}.$$

Here, if the parent of $\Phi.\texttt{x.next}$ were attached in the formation, the result of $\mathbb{D}(\Phi.\texttt{k.next})$ would not be equal to `43`. Instead, it would be equal to $\bot$, because $\Phi.\texttt{k.next}.\rho$ would still be attached to $\Phi.\texttt{x}$ after the dispatch of $\Phi.\texttt{k.next}$. The parent attribute may be compared with the `this` pointer in Java or C++, which does not point anywhere until a method of a class is called. Then, when the method is called, the `this` pointer refers to the formation that owns the method.

$$\mathbb{C}(\xi \triangleleft b) \to b \qquad \mathbb{C}(\Phi \triangleleft b) \to \Phi \qquad \mathbb{C}([\![B]\!] \triangleleft b) \to [\![B]\!] \qquad \mathbb{C}(\bot \triangleleft b) \to \bot$$
$$\mathbb{C}(e.\tau \triangleleft b) \to \mathbb{C}(e \triangleleft b).\tau \qquad \mathbb{C}(e_1(\tau \mapsto e_2) \triangleleft b) \to \mathbb{C}(e_1 \triangleleft b)(\tau \mapsto \mathbb{C}(e_2 \triangleleft b))$$

Fig. 2. Contextualization by induction.

5 Operators

In this section we introduce a family of semantic operators defined on ϕ-expressions. These operators do not extend the syntax of the calculus. Instead, they describe systematic transformations of expressions that reveal, modify, or project their semantic content. Each operator is defined in terms of the reduction semantics introduced earlier and preserves the core meaning of expressions while possibly changing their form or role.

5.1 Contextualization

The *contextualization* total function $\mathbb{C} : \mathcal{E} \times \mathcal{B} \to \mathcal{E}$ denoted as $\mathbb{C}(e \triangleleft b)$, which removes ξ from the expression, is defined by induction in Fig. 2.

5.2 Normalization

An expression that may be rewritten by the *rules* (or *reductions*) listed in Fig. 3 is a *reducible* expression. The notation $e_1 \rightsquigarrow e_2$, optionally followed by a condition, denotes a reduction of e_1 to e_2, if the condition holds.

These rules may be applied in any order.

A specific reduction may be denoted, for example, as $\rightsquigarrow_{\mathsf{R_{DOT}}}$, or just $\rightsquigarrow$ when no specific reduction is meant. The notation $e_1 \twoheadrightarrow e_2$ denotes a reflexive transitive closure of all reductions, so that there is a possibly empty finite sequence of reductions between e_1 and e_2.

Definition 5.1 (Normal Form). An expression that has no more possible applications of reductions is *irreducible* or a *normal form*, denoted as n_i ranging over $\mathcal{N} \subseteq \mathcal{E}$.

Thus, n is a normal form of e if $e \twoheadrightarrow n$ and there is no expression e_1 such that $n \rightsquigarrow e_1$.

Definition 5.2 (Absolute). A normal form is **absolute**, denoted by k and ranging over $\mathcal{K} \subseteq \mathcal{N}$, if it is either a) Φ, b) formation, c) dispatch with an absolute subject, or d) application with absolute subject and argument.

The *normalization* total function $\mathbb{N} : \mathcal{E} \to \mathcal{N}$ maps expressions to normal forms.

Section A demonstrates how normalization works through examples.

The reduction relation is *confluent*: if $e \twoheadrightarrow e_1$ and $e \twoheadrightarrow e_2$, then there exists an expression e_3 such that $e_1 \twoheadrightarrow e_3$ and $e_2 \twoheadrightarrow e_3$. Confluence guarantees that the normal form of an expression, when it exists, is unique, irrespective of the order in which

$$
\begin{array}{rl}
\mathsf{R}_{\text{ALPHA}}: & [\![B_1, \tau \mapsto \varnothing, B_2]\!](\alpha_i \mapsto e) \rightsquigarrow [\![B_1, \tau \mapsto \varnothing, B_2]\!](\tau \mapsto e) \quad \text{if } i = |\overline{B_1}| \\
\mathsf{R}_{\text{COPY}}: & [\![B_1, \tau \mapsto \varnothing, B_2]\!](\tau \mapsto k) \rightsquigarrow [\![B_1, \tau \mapsto k, B_2]\!] \\
\mathsf{R}_{\text{DC}}: & \bot(\tau \mapsto e) \rightsquigarrow \bot \\
\mathsf{R}_{\text{DCA}}: & \bot(\alpha_i \mapsto e) \rightsquigarrow \bot \\
\mathsf{R}_{\text{DD}}: & \bot.\tau \rightsquigarrow \bot \\
\mathsf{R}_{\text{DOT}}: & [\![B_1, \tau \mapsto n, B_2]\!].\tau \rightsquigarrow e(\rho \mapsto [\![B_1, \tau \mapsto n, B_2]\!]) \\
& \text{where } e := \mathbb{C}(n \triangleleft [\![B_1, \tau \mapsto n, B_2]\!]) \\
\mathsf{R}_{\text{MISS}}: & [\![B]\!](\tau \mapsto e) \rightsquigarrow \bot \quad \text{if } \tau \notin B \\
\mathsf{R}_{\text{NULL}}: & [\![B_1, \tau \mapsto \varnothing, B_2]\!].\tau \rightsquigarrow \bot \\
\mathsf{R}_{\text{OVER}}: & [\![B_1, \tau \mapsto e_1, B_2]\!](\tau \mapsto e_2) \rightsquigarrow \bot \quad \text{if } \tau \neq \rho \\
\mathsf{R}_{\text{PHI}}: & [\![B]\!].\tau \rightsquigarrow [\![B]\!].\varphi.\tau \quad \text{if } \varphi \in B \text{ and } \tau \notin B \\
\mathsf{R}_{\text{STAY}}: & [\![B_1, \rho \mapsto e_1, B_2]\!](\rho \mapsto e_2) \rightsquigarrow [\![B_1, \rho \mapsto e_1, B_2]\!] \\
\mathsf{R}_{\text{STOP}}: & [\![B]\!].\tau \rightsquigarrow \bot \quad \text{if } \tau \notin B \text{ and } \varphi \notin B \text{ and } \lambda \notin B
\end{array}
$$

Fig. 3. Reduction rules.

the rules are applied. We have proved this property mechanically in Lean 4, with the proof available in a public GitHub repository[3].

5.3 Equivalence

Definition 5.3 (Equivalence). Two expressions are said to be syntactically equivalent or just **equivalent** (denoted by $\equiv$) if their normal forms are syntactically identical.

5.4 Evaluation

The *evaluation* total function $\mathbb{E} : \mathcal{B} \times \mathcal{S} \to \mathcal{N} \times \mathcal{S}$ maps formations to normal forms by calling the function attached to λ and possibly modifying the *state* of evaluation:

$$\mathbb{E}([\![B_1, \lambda \mapsto F, B_2]\!], s_1) \to F([\![B_1, \lambda \mapsto F, B_2]\!], s_1) \to \langle n, s_2 \rangle.$$

The function F is called by value.

5.5 Morphing

The *morphing* partial function $\mathbb{M} : \mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightharpoonup \mathcal{N} \times \mathcal{S}$ maps normal forms to normal forms, possibly modifying the *state* of evaluation. The inference rules at Fig. 4 inductively describe the algorithm of morphing.

The notation $\langle n_1, e, s_1 \rangle \Downarrow \langle n_2, s_2 \rangle$ means that $\mathbb{M}(n_1, e, s_1)$ evaluates to n_2, thus *morphing* n_1 with a side-effect of changing s_1 to s_2, having Φ equal to e.

[3] https://github.com/objectionary/proof

$$
\begin{array}{rl}
\mathsf{R}_{\text{PRIM}}: & \mathbb{M}(\llbracket B \rrbracket) \rightsquigarrow \llbracket B \rrbracket \\
\mathsf{R}_{\text{LAMBDA}}: & \mathbb{M}(\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket.\tau) \rightsquigarrow \mathbb{M}(\mathcal{N}(e.\tau)) \\
& \text{where } e := \texttt{lambda}(\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket) \\
\mathsf{R}_{\text{DISPATCH}}: & \mathbb{M}(n.\tau) \rightsquigarrow \mathbb{M}(\mathcal{N}(e.\tau)) \\
& \text{where } e := \texttt{morph}(n) \\
\mathsf{R}_{\text{APPLICATION}}: & \mathbb{M}(n(\tau \mapsto e)) \rightsquigarrow \mathbb{M}(\mathcal{N}(e_1(\tau \mapsto e))) \\
& \text{where } e_1 := \texttt{morph}(n) \\
\mathsf{R}_{\text{APPLICATIONA}}: & \mathbb{M}(n(\alpha_i \mapsto e)) \rightsquigarrow \mathbb{M}(\mathcal{N}(e_1(\alpha_i \mapsto e))) \\
& \text{where } e_1 := \texttt{morph}(n) \\
\mathsf{R}_{\text{ROOT}}: & \mathbb{M}(\Phi) \rightsquigarrow \mathbb{M}(\mathcal{N}(e)) \qquad \text{if } e \neq \Phi \\
& \text{where } e := \texttt{global}() \\
\mathsf{R}_{\text{STUCK}}: & \mathbb{M}(n) \rightsquigarrow \bot
\end{array}
$$

Fig. 4. Morphing rules.

$$
\begin{array}{rl}
\mathsf{R}_{\text{DELTA}}: & \mathbb{D}(\llbracket B_1, \Delta \mapsto \delta, B_2 \rrbracket) \rightsquigarrow \delta \\
\mathsf{R}_{\text{BOX}}: & \mathbb{D}(\llbracket B_1, \varphi \mapsto e, B_2 \rrbracket) \rightsquigarrow \mathbb{D}(\mathcal{N}(e_1)) \qquad \text{if } [\Delta, \lambda] \cap (B_1 \cup B_2) = \emptyset \\
& \text{where } e_1 := \mathbb{C}(e \triangleleft \llbracket B_1, \varphi \mapsto e, B_2 \rrbracket) \\
\mathsf{R}_{\text{FIRE}}: & \mathbb{D}(\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket) \rightsquigarrow \mathbb{D}(\mathcal{N}(e)) \\
& \text{where } e := \texttt{lambda}(\llbracket B_1, \lambda \mapsto F, B_2 \rrbracket) \\
\mathsf{R}_{\text{NONE}}: & \mathbb{D}(\llbracket B \rrbracket) \rightsquigarrow \varnothing \\
\mathsf{R}_{\text{BOTT}}: & \mathbb{D}(\bot) \rightsquigarrow \varnothing \\
\mathsf{R}_{\text{NORM}}: & \mathbb{D}(n) \rightsquigarrow \mathbb{D}(\mathbb{M}(n))
\end{array}
$$

Fig. 5. Dataization rules.

5.6 Dataization

The *dataization* partial function $\mathbb{D} : \mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightharpoonup \mathcal{D} \times \mathcal{S}$ maps normal forms to data, possibly modifying the *state* of evaluation. The inference rules in Fig. 5 inductively describe the algorithm of dataization.

The notation $\langle n,\ e,\ s_1 \rangle \Downarrow \langle \delta,\ s_2 \rangle$ means that $\mathbb{D}(n, e, s_1)$ evaluates to δ, thus *dataizing* n with a side-effect of changing the state of evaluation s_1 to s_2, having Φ equal to e. The notation $\mathbb{D}(n, u)$ is a shortened form of $\mathbb{D}(n, u, s_0)_{(1)}$, which is a *pure* dataization — inputting an empty state of evaluation and ignoring the output state of evaluation. Here and later, the notation $x_{(i)}$ denotes the i-th element of the tuple x (counting starts from one).

Section B demonstrates how the dataization function works through examples.

5.7 Congruence

Definition 5.4 (Congruence). Two expressions n_1 and n_2 are said to be behaviorally equivalent or **congruent** (denoted by $\cong$) if for any state s and any e: $\mathbb{D}(n_1, e, s) = \mathbb{D}(n_2, e, s)$.

Two congruent expressions may be non-equivalent, for example:

$$\left\{\tau_1 \mapsto [\![\texttt{foo} \mapsto \varnothing, \Delta \mapsto \texttt{01-02}]\!], \tau_2 \mapsto [\![\texttt{bar} \mapsto \varnothing, \Delta \mapsto \texttt{01-02}]\!]\right\}$$
$$\Phi.\tau_1 \cong \Phi.\tau_2 \not\Rightarrow \Phi.\tau_1 \equiv \Phi.\tau_2.$$

6 Related Work

In this section we analyze and categorize prior art related to our work. Neither object-oriented formalism nor pure object-oriented languages are new research topics. However, we identified certain gaps in existing studies that make us believe that our work has novelty.

Attempts were made to formalize OOP and introduce an object calculus similar to the lambda calculus [**?**] used in functional programming. For example, **?**] suggested an imperative calculus of objects, which was extended by **?**] to support classes, by **?**] to support concurrency and synchronization, and by **?**] to support distributed programming.

Earlier, **?**] combined OOP and π-calculus in order to introduce object calculus for asynchronous communication, which was further referenced by **?**] in their work on object-based design notation.

A few attempts were made to reduce existing OOP languages and formalize what is left. Featherweight Java is the most notable example proposed by **?**], which omits almost all features of the full language (including interfaces and even assignment) to obtain a small calculus. Later it was extended by **?**] to support nested and multi-threaded transactions. Featherweight Java is used in formal languages such as Obsidian [**?**] and SJF [**?**].

Another example is Larch/C++ [**?**], which is a formal algebraic interface specification language tailored to C++. It allows interfaces of C++ classes and functions to be documented in a way that is unambiguous and concise.

Several attempts to formalize OOP were made by extensions of the most popular formal notations and methods, such as Object-Z [**?**] and VDM++ [**?**]. In Object-Z, state and operation schemes are encapsulated into classes. The formal model is based upon the idea of a class history [**?**]. However, all these OO extensions do not have comprehensive refinement rules that can be used to transform specifications into implemented code in an actual OO programming language, as was noted by **?**].

?] suggested an object calculus as an extension to relational calculus. **?**] further developed these ideas and related them to a Boolean algebra. **?**] developed an algorithm to transform an object calculus into an object algebra.

However, all these theoretical attempts to formalize OO languages were unable to fully describe their features, as was noted by **?**]: "The development of concurrent object-based programming languages has suffered from the lack of any generally accepted formal foundations for defining their semantics." In addition, when describing the attempts of formalization, **?**] summarized: "Not one of the notations is defined formally, nor provided with denotational semantics, nor founded on axiomatic semantics." Moreover, despite these efforts, **? ? ?**] noted in their series of works that

relatively little formal work has been carried out on object-based languages, and it remains true to this day.

7 Future Studies

This paper intentionally focuses on defining φ-calculus (syntax, foundations, and semantic operators) and does not yet demonstrate practical use. At the same time, it motivates the calculus as a step toward more rigorous reasoning, verified compilers and static analyzers, and better language design. Future work may build on φ-calculus by creating the following artifacts:

- A short list of core laws about reductions (what always works, what may fail).
- A clear way to prove that two expressions behave the same in any context.
- A type system that catches mistakes early.
- A way to mark which expressions are pure and which may have side effects.
- A catalog of safe rewrites (how to change code without changing meaning).
- A translation from Java or C++ into φ-expressions.
- A standard library of built-in atoms with precise rules.
- Larger language features built as add-ons (overriding, interfaces, constructors).
- Add-ons for mutation, concurrency, and distribution, with simple safety guarantees.

We invite readers to extend, formalize, and apply φ-calculus to real object-oriented patterns and languages, and to help turn these foundations into a shared body of results and artifacts.

8 Acknowledgments

Many thanks to (in alphabetical order of last names) Aliaksei Bialíauski, Fabricio Cabral, Kirill Chernyavskiy, Piotr Chmielowski, Danilo Danko, Konstantin Gukov, Andrew Graur, Ali-Sultan Kigizbaev, Nikolai Kudasov, Alexander Legalov, Mikhail Lipanin, Tymur λysenko, Alexandr Naumchev, Alonso A. Ortega, John Page, Alex Panov, Maxim Petrov, Alexander Pushkarev, Marcos Douglas B. Santos, Alex Semenyuk, Violetta Sim, Sergei Skliar, Stian Soiland-Reyes, Viacheslav Tradunskyi, Ilya Trub, César Soto Valero, Alena Vasileva, David West, Vladimir Zakharov, and Evgeny Zouev for their contribution to the development of φ-calculus.

A Examples of Normalization

The following examples demonstrate how the reduction rules of Fig. 3 may normalize some expressions, involving the contextualization function (Section 5.1):

$$
\begin{aligned}
e_1 &= \overbrace{\llbracket \mathtt{x} \mapsto \mathtt{t},\ \mathtt{t} \mapsto \varnothing \rrbracket}^{\Pi_1}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_1.\mathtt{t}(\rho \mapsto \Pi_1) \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \bot(\rho \mapsto \Pi_1) \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_2 &= \llbracket \mathtt{x} \mapsto \overbrace{\llbracket \mathtt{t} \mapsto 42 \rrbracket}^{\Pi_2}.\mathtt{t} \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow 42.
\end{aligned}
$$

$$
\begin{aligned}
e_3 &= \llbracket \mathtt{x} \mapsto \varnothing \rrbracket(\alpha_1 \mapsto 42).\mathtt{x} \rightsquigarrow_{\mathsf{R_{ALPHA}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \varnothing \rrbracket(\rho \mapsto 42).\mathtt{x} \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \varnothing,\ \rho \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_4 &= \llbracket \mathtt{x} \mapsto \varnothing \rrbracket(42).\mathtt{x} \rightsquigarrow_{\mathsf{R_{ALPHA}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \varnothing \rrbracket(\mathtt{x} \mapsto 42).\mathtt{x} \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow 42.
\end{aligned}
$$

$$
\begin{aligned}
e_5 &= \llbracket \mathtt{x} \mapsto \llbracket \lambda \mapsto \mathtt{Fn} \rrbracket.\rho.\mathtt{k},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\mathtt{k},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto \bot,\ \mathtt{k} \mapsto 42 \rrbracket}^{\Pi_3}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \bot(\rho \mapsto \Pi_3) \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_6 &= \llbracket \mathtt{x} \mapsto \varnothing,\ \mathtt{y} \mapsto \mathtt{x} \rrbracket(\mathtt{x} \mapsto 42).\mathtt{y} \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto 42,\ \mathtt{y} \mapsto \mathtt{x} \rrbracket}^{\Pi_4}.\mathtt{y} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_4.\mathtt{x}(\rho \mapsto \Pi_4) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow 42.
\end{aligned}
$$

$$
\begin{aligned}
e_7 &= \llbracket \mathtt{x} \mapsto \mathtt{t},\ \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto 42) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \mathtt{t},\ \mathtt{t} \mapsto 42 \rrbracket.
\end{aligned}
$$

$$
\begin{aligned}
e_8 &= \overbrace{\llbracket \mathtt{x} \mapsto \mathtt{t},\ \varphi \mapsto \llbracket \mathtt{t} \mapsto \llbracket\rrbracket \rrbracket}^{\Pi_5}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_5.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{PHI}}} \\
&\rightsquigarrow \Pi_5.\varphi.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{t} \mapsto \llbracket\rrbracket \rrbracket(\rho \mapsto \Pi_5).\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \overbrace{\llbracket \mathtt{t} \mapsto \llbracket\rrbracket,\ \rho \mapsto \Pi_5 \rrbracket}^{\Pi_6}.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket\rrbracket(\rho \mapsto \Pi_6,\ \rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \rho \mapsto \Pi_6 \rrbracket(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \llbracket \rho \mapsto \Pi_6 \rrbracket.
\end{aligned}
$$

$$
\begin{aligned}
e_9 &= \overbrace{\llbracket \mathtt{x} \mapsto \rho.\rho.\mathtt{t} \rrbracket}^{\Pi_7}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_7.\rho.\rho.\mathtt{t}(\rho \mapsto \Pi_7) \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \bot.\rho.\mathtt{t}(\rho \mapsto \Pi_7) \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \bot.\mathtt{t}(\rho \mapsto \Pi_7) \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \bot(\rho \mapsto \Pi_7) \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_{10} &= \llbracket \mathtt{x} \mapsto \llbracket\rrbracket.\rho.\mathtt{k},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\mathtt{k},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto \bot,\ \mathtt{k} \mapsto 42 \rrbracket}^{\Pi_8}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \bot(\rho \mapsto \Pi_8) \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_{11} &= \overbrace{\llbracket \mathtt{x} \mapsto \mathtt{t}(\mathtt{k} \mapsto \mathtt{f}).\mathtt{k},\ \mathtt{t}(\mathtt{k}) \mapsto \llbracket\rrbracket,\ \mathtt{f} \mapsto \llbracket\rrbracket \rrbracket}^{\Pi_9}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_9.\mathtt{t}(\mathtt{k} \mapsto \Pi_9.\mathtt{f}).\mathtt{k}(\rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{k} \mapsto \varnothing \rrbracket(\rho \mapsto \Pi_9,\ \mathtt{k} \mapsto \llbracket\rrbracket(\rho \mapsto \Pi_9)).\mathtt{k}(\rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{k} \mapsto \varnothing,\ \rho \mapsto \Pi_9 \rrbracket(\mathtt{k} \mapsto \overbrace{\llbracket \rho \mapsto \Pi_9 \rrbracket}^{\Pi_{10}}).\mathtt{k}(\rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{k} \mapsto \Pi_{10},\ \rho \mapsto \Pi_9 \rrbracket.\mathtt{k}(\rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_{10}(\rho \mapsto \llbracket \mathtt{k} \mapsto \Pi_{10},\ \rho \mapsto \Pi_9 \rrbracket,\ \rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \Pi_{10}(\rho \mapsto \Pi_9) \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \Pi_{10}.
\end{aligned}
$$

$$
\begin{aligned}
e_{12} &= \overset{\Pi_{11}}{\overbrace{\llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto \mathtt{k}),\ \mathtt{k} \mapsto \llbracket \lambda \mapsto \mathtt{Fn} \rrbracket \rrbracket}}.\mathtt{x}.\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \llbracket \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto \Pi_{11}.\mathtt{k},\ \rho \mapsto \Pi_{11}).\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \llbracket \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto \llbracket \lambda \mapsto \mathtt{Fn} \rrbracket(\rho \mapsto \Pi_{11}),\ \rho \mapsto \Pi_{11}).\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \llbracket \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto \overset{\Pi_{12}}{\overbrace{\llbracket \lambda \mapsto \mathtt{Fn},\ \rho \mapsto \Pi_{11} \rrbracket}},\ \rho \mapsto \Pi_{11}).\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \llbracket \mathtt{t} \mapsto \Pi_{12} \rrbracket(\rho \mapsto \Pi_{11}).\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \overset{\Pi_{13}}{\overbrace{\llbracket \mathtt{t} \mapsto \Pi_{12},\ \rho \mapsto \Pi_{11} \rrbracket}}.\mathtt{t}.\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \Pi_{12}(\rho \mapsto \Pi_{13}).\mathtt{p} \rrbracket \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \llbracket \mathtt{a} \mapsto \Pi_{12}.\mathtt{p} \rrbracket.
\end{aligned}
$$

$$
\begin{aligned}
e_{13} &= \llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket.\mathtt{p} \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket.\rho.\rho.\mathtt{k}(\rho \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket) \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{NULL}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \bot.\rho.\mathtt{k}(\rho \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket) \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \bot.\mathtt{k}(\rho \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket) \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DD}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \bot(\rho \mapsto \llbracket \mathtt{p} \mapsto \rho.\rho.\mathtt{k} \rrbracket) \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \bot \rrbracket.\mathtt{t},\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{x} \mapsto \bot(\rho \mapsto \llbracket \mathtt{t} \mapsto \bot \rrbracket),\ \mathtt{k} \mapsto 42 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \overset{\Pi_{14}}{\overbrace{\llbracket \mathtt{x} \mapsto \bot,\ \mathtt{k} \mapsto 42 \rrbracket}}.\mathtt{x} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \bot(\rho \mapsto \Pi_{14}) \rightsquigarrow_{\mathsf{R_{DC}}} \\
&\rightsquigarrow \bot.
\end{aligned}
$$

$$
\begin{aligned}
e_{14} &= \overset{\Pi_{15}}{\overbrace{\llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \rho.\mathtt{k}.\rho.\mathtt{t} \rrbracket,\ \mathtt{k} \mapsto [\![]\!],\ \mathtt{t} \mapsto [\![]\!] \rrbracket}}.\mathtt{x}.\mathtt{t} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \llbracket \mathtt{t} \mapsto \overset{\Pi_{16}}{\overbrace{\rho.\mathtt{k}.\rho.\mathtt{t}}} \rrbracket(\rho \mapsto \Pi_{15}).\mathtt{t} \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \overset{\Pi_{17}}{\overbrace{\llbracket \mathtt{t} \mapsto \Pi_{16},\ \rho \mapsto \Pi_{15} \rrbracket}}.\mathtt{t} \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_{17}.\rho.\mathtt{k}.\rho.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_{15}(\rho \mapsto \Pi_{17}).\mathtt{k}.\rho.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \overset{\Pi_{18}}{\overbrace{\llbracket \mathtt{x} \mapsto \llbracket \mathtt{t} \mapsto \Pi_{16} \rrbracket,\ \mathtt{k} \mapsto [\![]\!],\ \mathtt{t} \mapsto [\![]\!],\ \rho \mapsto \Pi_{17} \rrbracket}}.\mathtt{k}.\rho.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow [\![]\!](\rho \mapsto \Pi_{18}).\rho.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \overset{\Pi_{19}}{\overbrace{\llbracket \rho \mapsto \Pi_{18} \rrbracket}}.\rho.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow \Pi_{18}(\rho \mapsto \Pi_{19}).\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \Pi_{18}.\mathtt{t}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{DOT}}} \\
&\rightsquigarrow [\![]\!](\rho \mapsto \Pi_{18},\ \rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{COPY}}} \\
&\rightsquigarrow \Pi_{19}(\rho \mapsto \Pi_{17}) \rightsquigarrow_{\mathsf{R_{STAY}}} \\
&\rightsquigarrow \Pi_{19}.
\end{aligned}
$$

The following expressions may not be reduced any further; they are in normal form:

$$
e_{15} = \llbracket \mathtt{x} \mapsto \varnothing \rrbracket(\mathtt{x} \mapsto \mathtt{t}).
$$

$$e_{16} = ⟦ \mathtt{x} \mapsto \mathtt{k},\ \mathtt{k} \mapsto 42 ⟧.$$

$$e_{17} = ⟦ \mathtt{x} \mapsto \mathtt{t},\ \lambda \mapsto \mathtt{Fn} ⟧.$$

$$e_{18} = ⟦ \mathtt{x} \mapsto \mathtt{k},\ \mathtt{t} \mapsto 42 ⟧.$$

$$e_{19} = ⟦ \mathtt{k} \mapsto ⟦ \mathtt{x} \mapsto 42,\ \lambda \mapsto \mathtt{Fn} ⟧.\mathtt{y} ⟧.$$

$$e_{20} = ⟦ \mathtt{x} \mapsto ⟦ \mathtt{t} \mapsto \Phi.\mathtt{x} ⟧ ⟧.$$

Normalization of these expressions leads to endless recursion:

$$\begin{aligned}
e_{21} &= \overbrace{⟦ \mathtt{x} \mapsto \mathtt{y},\ \mathtt{y} \mapsto \mathtt{x} ⟧}^{\Pi_{20}}.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \overbrace{\Pi_{20}.\mathtt{y}(\rho \mapsto \Pi_{20})}^{\Pi_{21}} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{20}.\mathtt{x}(\rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \overbrace{\Pi_{21}(\rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20})}^{\Pi_{22}} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{20}.\mathtt{x}(\rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20}) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\rightsquigarrow \Pi_{22}(\rho \mapsto \Pi_{20},\ \rho \mapsto \Pi_{20}) \rightsquigarrow \\
&\rightsquigarrow \ldots
\end{aligned}$$

B Example of Dataization

Consider a program that converts degrees from Celsius to Fahrenheit using the formula:

$$^\circ F = {}^\circ C \times 1.8 + 32,$$

Assuming that $^\circ C$ already equals `25` and is attached to the `c` attribute instead of being provided by a user:

$$\left\{\llbracket
\varphi \mapsto \llbracket \varphi \mapsto \texttt{c.times(1.8).plus(32)}, \texttt{c} \mapsto 25 \rrbracket,
\texttt{bytes(data)} \mapsto \llbracket \varphi \mapsto \texttt{data} \rrbracket,
\texttt{number(as-bytes)} \mapsto \llbracket \varphi \mapsto \texttt{as-bytes}, \texttt{times(x)} \mapsto \llbracket \lambda \mapsto \texttt{F1} \rrbracket, \texttt{plus(x)} \mapsto \llbracket \lambda \mapsto \texttt{F2} \rrbracket \rrbracket
\rrbracket\right\}.$$

The functions are defined as follows:

$$\texttt{F1} : \langle b, e, s \rangle \to \langle e.\texttt{number}(\Delta \mapsto \mathbb{D}(b.\rho, e) \times \mathbb{D}(b.\texttt{x}, e)), s \rangle$$
$$\texttt{F2} : \langle b, e, s \rangle \to \langle e.\texttt{number}(\Delta \mapsto \mathbb{D}(b.\rho, e) + \mathbb{D}(b.\texttt{x}, e)), s \rangle$$

The following sequence of transformations constitutes the evaluation of the program:

φ-Calculus: Object-Oriented Formalism

$$
\begin{aligned}
&\overset{\Pi_{01}}{[\varphi \mapsto \texttt{c.times(1.0).plus(32)},\ \rho \mapsto \texttt{25}]} \rightsquigarrow_{\mathrm{C}} \\
&\rightsquigarrow \Pi_{01}[\texttt{c.times(1.0).plus(32)} \rightsquigarrow_{\mathrm{Application}}] \\
&\rightsquigarrow \overset{\Pi_{01}}{[\texttt{25.times(1.0).plus(32)} \rightsquigarrow_{\mathrm{Application}}]} \\
&\rightsquigarrow [\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F1}, \rho \mapsto [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-46-80-00-00-00-00-00}])], \varphi \mapsto \texttt{as-bytes.times} \mapsto \overset{\Pi_{04}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F2}]}.\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F3}], \rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte}(\texttt{data}) \mapsto [\varphi \mapsto \texttt{data}].\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F4}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}]]]]](32) \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F1}, \rho \mapsto [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-46-80-00-00-00-00-00}])], \varphi \mapsto \texttt{as-bytes.times} \mapsto \overset{\Pi_{04}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F2}]}.\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F3}], \rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte}(\overset{\Pi_{06}}{\texttt{data} \mapsto \varnothing, \varphi \mapsto \texttt{data}}).\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F4}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}]]]](32) \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F1}, \rho \mapsto [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-46-80-00-00-00-00-00}]), \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{04}.\texttt{plus}(\texttt{x}) \mapsto \overset{\Pi_{05}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F3}]}.\rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F4}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}]]]](32) \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{x} \mapsto 32, \lambda \mapsto \texttt{F1}, \rho \mapsto [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-46-80-00-00-00-00-00}]), \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{04}.\texttt{plus}(\texttt{x}) \mapsto \overset{\Pi_{05}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F3}]}.\rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F4}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}]]]](32) \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{x} \mapsto 32, \lambda \mapsto \texttt{F1}, \rho \mapsto [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-46-80-00-00-00-00-00}]), \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{04}.\texttt{plus}(\texttt{x}) \mapsto \overset{\Pi_{05}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F3}]}.\rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F4}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}]]]](32) \rightsquigarrow_{\mathrm{Alpha}} \\
&77 \rightsquigarrow_{\mathrm{Application}} \\
&\rightsquigarrow [\texttt{as-bytes} \mapsto \varnothing, \varphi \mapsto \texttt{as-bytes.times} \mapsto \overset{\Pi_{04}}{[\texttt{x} \mapsto \varnothing, \lambda \mapsto \texttt{F1}]}.\texttt{plus} \mapsto \Pi_{04}, \rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{bytes} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}[\Pi_{07}[\varnothing.\texttt{byte}([\Delta \mapsto \texttt{40-53-40-00-00-00-00-00}]).\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-53-40-00-00-00-00-00}])].\texttt{plus} \mapsto \Pi_{07}]].\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-53-40-00-00-00-00-00}])] \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{as-bytes} \mapsto \varnothing, \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{04}.\texttt{plus} \mapsto \Pi_{04}, \rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{bytes} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}]].\texttt{as-bytes} \mapsto \Phi.\texttt{byte}([\Delta \mapsto \texttt{40-53-40-00-00-00-00-00}])] \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\texttt{as-bytes} \mapsto \Phi.\texttt{byte}(\Pi_{09}), \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{04}.\texttt{plus} \mapsto \overset{\Pi_{05}}{[\varphi \mapsto \Pi_{05}.\texttt{byte} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}]]}, \rho \mapsto [\varphi \mapsto \Pi_{05}.\texttt{byte} \mapsto \Pi_{07}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}]]] \rightsquigarrow_{\mathrm{C}} \\
&\rightsquigarrow \Pi_{09}[\texttt{as-bytes} \mapsto \Pi_{09}, \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{09}, \rho \mapsto [\varphi \mapsto \Pi_{09}.\texttt{bytes} \mapsto \Pi_{09}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}]]] \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow \Pi_{09}(\rho \mapsto \Pi_{09}) \rightsquigarrow_{\mathrm{Application}} \\
&\rightsquigarrow \texttt{data} \mapsto \Pi_{09}, \varphi \mapsto \texttt{data}.\rho \mapsto [\varphi \mapsto \Pi_{09}.\texttt{byte} \mapsto \Pi_{09}.\texttt{number}(\overset{\Pi_{09}}{\texttt{as-bytes} \mapsto [\varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{09}]}](\rho \mapsto [\texttt{as-bytes} \mapsto \Pi_{09}, \varphi \mapsto \texttt{as-bytes.times} \mapsto \Pi_{09}.\texttt{plus} \mapsto \Pi_{07}, \rho \mapsto [\varphi \mapsto \Pi_{09}.\texttt{bytes} \mapsto \Pi_{09}.\texttt{number}(\texttt{as-bytes}) \mapsto [\varphi \mapsto \texttt{as-bytes.times}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F5}].\texttt{plus}(\texttt{x}) \mapsto [\lambda \mapsto \texttt{F6}]]]])] \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow \Pi_{09}(\rho \mapsto \Pi_{09}) \rightsquigarrow_{\mathrm{Application}} \\
&\rightsquigarrow \Pi_{09} \rightsquigarrow_{\mathrm{C}} \\
&\rightsquigarrow \Pi_{09} \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow \Pi_{09}[\texttt{data} \mapsto \Pi_{09}] \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow \Pi_{09}(\rho \mapsto \Pi_{09}) \rightsquigarrow_{\mathrm{Alpha}} \\
&\rightsquigarrow [\Delta \mapsto \texttt{40-53-40-00-00-00-00-00}, \rho \mapsto \Pi_{09}].
\end{aligned}
$$